

Penilaian Kualitas Gambar dengan *Edge Detection* untuk *Image Selection* menggunakan OpenCV

1st Muhammad Irsyad Asyarie
Teknik Robotika dan Kecerdasan Buatan
Universitas Airlangga
Surabaya, Indonesia
muhammad.syad.asyarie-
2021@ftmm.unair.ac.id

2nd Muhammad Izza Dwi N. A
Teknik Robotika dan Kecerdasan Buatan
Universitas Airlangga
Surabaya, Indonesia
line 5: email address or ORCID

3rd Maharani Yassar Dewanti
Teknik Robotika dan Kecerdasan Buatan
Universitas Airlangga
Surabaya, Indonesia
maharani.yassar.dewanti-
2021@ftmm.unair.ac.id

Abstract— As time goes by, it becomes easier for humans to take pictures. Therefore, an image processing system is needed to assess the quality of the images obtained. Digital image processing is a branch of science that is currently being developed to make human life easier. One of them is edge detection, which is widely used for various applications involving image processing, such as computer vision, medical image processing, and visual analysis. There are many Edge Detection methods that have been developed by experts. Each has its own advantages and disadvantages. In this article the author uses the Laplace Operator and Canny Operator Methods to assess the quality of PCB images captured using a microscope with different lens scales. All these images have been collected into one image dataset.

The aim of this research is to create an image processing system on a machine so that the machine can judge which images are of higher quality, the same way humans can differentiate with the ordinary eye. The author examines and implements Edge Detection using the Laplace Operator and Canny Operator methods and then displays the results of executing these methods on several images.

Keywords— *Digital Image, Edge Detection, Grayscale Image Conversion, Canny Operator, Laplace Operator, OpenCV, Image Quality Assessment, Image Selection, Image Resolution.*

I. PENDAHULUAN

Gambar merupakan media yang sangat penting untuk mengkomunikasikan informasi dan ide. Pemilihan gambar yang berkualitas merupakan langkah kritis dalam berbagai aplikasi yang melibatkan pengolahan gambar, seperti visi komputer, pemrosesan citra medis, dan analisis visual. Bagi peneliti, profesional, dan pengembang perangkat lunak, kemampuan untuk secara otomatis menilai kualitas gambar menjadi semakin penting seiring dengan meningkatnya volume gambar yang dihasilkan oleh berbagai perangkat dan sumber. Untuk mengatasi tantangan ini, teknik pemrosesan citra yang canggih diperlukan untuk mengidentifikasi dan memilih gambar-gambar berkualitas tinggi.

Salah satu pendekatan yang umum digunakan dalam penilaian kualitas gambar adalah metode analisis Edge Detection. Edge Detection adalah teknik yang digunakan untuk menemukan perubahan tajam dalam intensitas piksel gambar, yang sering kali merupakan indikator kualitas gambar. Semakin tajam tepi yang dihasilkan, semakin baik kualitas gambar tersebut. Edge detection adalah salah satu algoritma yang umum digunakan untuk meningkatkan kualitas gambar dengan menghilangkan piksel-piksel yang tidak penting dan memperkuat kontur-kontur pada gambar.

Dalam konteks ini, penggunaan perangkat lunak OpenCV (Open Source Computer Vision Library) menjadi sangat relevan. OpenCV adalah perpustakaan sumber terbuka yang kuat dan serbaguna yang menyediakan berbagai alat untuk pemrosesan gambar dan penglihatan komputer. Kami memanfaatkan keunggulan OpenCV untuk mengevaluasi kualitas gambar dengan menggabungkan metode penilaian kualitas gambar dengan deteksi tepi gambar. Penelitian ini bertujuan untuk mengembangkan algoritma yang efektif untuk menilai kualitas gambar dan memilih gambar dengan kualitas terbaik dari suatu kumpulan gambar.

II. LANDASAN TEORI DAN METODOLOGI

A. OpenCV

OpenCV (Open Source Computer Vision Library) adalah sebuah library open source yang dikembangkan oleh Intel yang fokus untuk menyederhanakan programming terkait citra digital[1]. OpenCV merupakan open source computer vision library untuk bahasa pemrograman C/C++, dan telah dikembangkan ke Python, Java, Matlab[1][2]. Library dapat diunduh melalui link <http://sourceforge.net/projects/opencvlibrary> atau melalui <http://opencv.org/releases.html>. Saat ini versi yang terbaru adalah 4.1.2 dengan tanggal rilis 10 Oktober 2019[2].

OpenCV memiliki banyak fitur yang dapat dimanfaatkan, antara lain: pengenalan wajah, pelacakan wajah, deteksi wajah, Kalman filtering, dan berbagai jenis metode AI (Artificial Intelligence)[1]. Di dalam OpenCV juga termasuk kemampuan “high level”, seperti kemampuan tambahan untuk deteksi wajah, pengenalan wajah, termasuk optical flow - Metode untuk AI dan machine learning[2]. OpenCV mendukung multiplatform, dapat mendukung baik Windows ataupun Linux, dan sekarang telah mendukung MacOSX dan Android[1][2]. Beberapa fitur utama dari OpenCV antara lain[1][2]:

a. Image dan Video I/O

OpenCV menyediakan alat untuk membaca dan menulis gambar dan video dari berbagai sumber dan format file. Ini termasuk dukungan untuk format gambar seperti JPEG, PNG, BMP, dan format video seperti AVI, MP4, dan banyak lainnya. Fungsi I/O OpenCV memudahkan pemrosesan dan analisis gambar yang berasal dari berbagai sumber, termasuk perangkat kamera dan file.

b. Algoritma image processing dan Vision

OpenCV menghadirkan beragam algoritma pengolahan gambar dan penglihatan komputer, termasuk deteksi tepi, perubahan ukuran, filtrasi, peningkatan citra, segmentasi objek, dan pemrosesan stereo 3D. Algoritma pengolahan gambar ini memungkinkan Anda untuk memanipulasi dan menganalisis citra dengan berbagai cara, termasuk ekstraksi fitur, pelacakan objek, pencocokan gambar, dan banyak lagi.

c. Machine learning library

OpenCV memiliki modul pembelajaran mesin yang memungkinkan pelatihan dan penggunaan model pembelajaran mesin untuk tugas-tugas seperti klasifikasi dan deteksi objek. Ini termasuk alat untuk bekerja dengan berbagai jenis model pembelajaran mesin, termasuk mesin vektor pendukung (SVM), jaringan saraf tiruan, pohon keputusan, dan lainnya.

d. GUI, image, dan video I/O

OpenCV menyediakan fungsi GUI yang memungkinkan Anda untuk membuat aplikasi berbasis GUI untuk pemrosesan gambar dan penglihatan komputer. Ini juga memungkinkan pengembangan antarmuka pengguna dengan fungsi untuk menampilkan gambar, menggambar tanda, dan berinteraksi dengan perangkat input seperti mouse dan keyboard.

e. Struktur data, support XML dan fungsi-fungsi grafis

OpenCV mencakup struktur data untuk merepresentasikan objek seperti titik, garis, poligon, dan kurva. Ini memudahkan representasi dan manipulasi objek di citra. OpenCV juga mendukung berbagai format XML, yang dapat digunakan untuk menyimpan dan membaca konfigurasi, pelatihan model, dan data lainnya. Fungsi grafis di OpenCV memungkinkan Anda untuk menggambar berbagai bentuk, teks, dan anotasi pada gambar, yang berguna untuk tujuan pemrosesan dan analisis.

OpenCV dapat digunakan dalam berbagai macam bahasa pemrograman, seperti C++, Python, Java, dan lain sebagainya[2]. OpenCV juga dapat melakukan operasi high-speed GPU dengan menggunakan antarmuka-antarmuka berbasis CUDA dan OpenCL[1]. OpenCV memiliki banyak sekali fitur yang dapat dimanfaatkan, sehingga dapat digunakan dalam berbagai aplikasi, seperti pengolahan gambar, pengenalan objek, dan lain sebagainya[1][2].

B. Citra Digital

Citra digital merupakan sebuah fungsi intensitas cahaya $f(x,y)$, dimana nilai x dan y merupakan koordinat spasial dan nilai fungsi tersebut pada setiap titik (x,y) merupakan tingkat kecermerlangan citra pada titik tersebut. Citra digital dinyatakan dengan matrik berukuran $N \times M$ (N menyatakan baris atau tinggi, M menyatakan kolom atau lebar) [4]. Ukurannya dinyatakan dalam titik atau piksel (pixel = picture element) dan dapat pula dinyatakan dalam satuan panjang (mm atau inci = inch).

$$f(x,y) \approx \begin{bmatrix} f(0,0) & f(0,1) & \dots & f(0,M-1) \\ f(1,0) & f(1,1) & \dots & f(1,M-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(N-1,0) & f(N-1,1) & \dots & f(N-1,M-1) \end{bmatrix}$$

Keterangan:

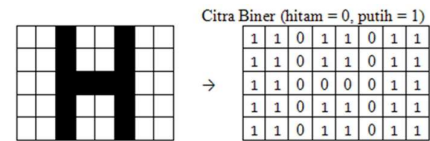
N = jumlah baris, $0 < y < N - 1$

M = jumlah baris, $0 < y < M - 1$

L = maksimal warna intensitas (derajat keabuan),
 $0 < f(x,y) < L - 1$

Berdasarkan format penyimpanan nilai warnanya, citra terdiri atas empat jenis [4]:

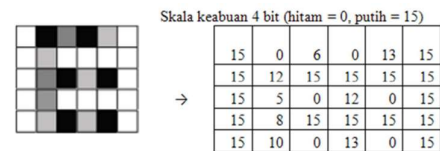
1. Citra biner atau monokrom



Gambar 2.1 Citra Asli

Pada citra jenis ini, setiap titik atau piksel hanya bernilai 0 atau 1.

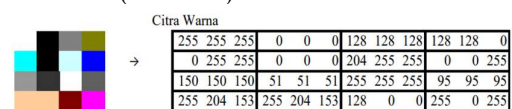
2. Citra skala keabuan



Gambar 2.2 Citra Skala Keabuan

Citra skala keabuan mempunyai kemungkinan warna antara hitam (minimal) dan putih (maksimal). Suatu citra dengan skala keabuan 4 bit, memiliki jumlah kemungkinan warna 2 4 16 warna.

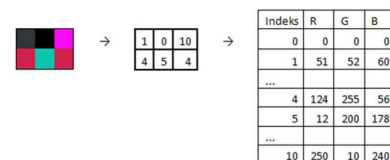
3. Citra warna (true color)



Gambar 2.3 Citra Warna

Setiap titik (piksel) pada citra warna mewakili warna yang merupakan kombinasi dari tiga warna dasar yaitu merah, hijau, dan biru yang dikenal sebagai citra RGB (Red, Green, Blue). Setiap warna dasar memiliki intensitas sendiri dengan nilai maksimum 255 (8 bit).

4. Citra warna berindeks



Gambar 2.4 Citra Warna Berindeks

Setiap titik (piksel) pada citra warna berindeks mewakili indeks dari suatu tabel warna yang tersedia (biasanya disebut palet warna).

C. Konversi Citra Warna (RGB) ke Citra Skala Keabuan (Grayscale)

Proses pertama dari metode pendeteksi citra secara umum termasuk metode Roberts' Cross adalah melakukan konversi

citra, dari citra berwarna ke citra skala keabuan (grayscale). Rumus yang digunakan untuk konversi adalah sebagai berikut:

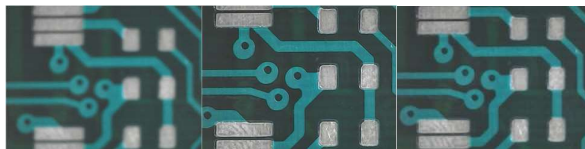
$$x = \frac{(R+G+B)}{3}$$

R adalah nilai warna merah (Red), G adalah nilai warna hijau (Green), dan B adalah nilai warna biru (Blue) dari pixel yang diproses.

D. Resolusi Citra

Resolusi citra adalah banyaknya titik untuk setiap satuan panjang (dot per inch) yang terdapat dalam citra tersebut. Semakin besar resolusi makin banyak titik yang terkandung dalam citra, sehingga menjadi lebih halus dalam visualisasinya dan semakin berkualitas[4]. Menurut kualitas resolusi citra yang dihasilkan, resolusi dibedakan [7]:

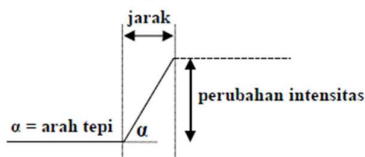
1. Resolusi tinggi (high resolution), jika tingkat ketelitian yang cukup tinggi dari suatu media dalam menangkap ataupun menampilkan datanya.
2. Resolusi rendah (low resolution), jika pada gambar atau layar yang teks dan grafiknya tampil dengan detail yang relatif kasar.



Gambar 2.5 Citra dengan Beberapa Tingkat Resolusi

E. Edge Detection

Edge dapat didefinisikan sebagai batas antara dua region (dua piksel yang saling berdekatan) yang memiliki perbedaan intensitas yang tajam atau tinggi [4]. Edge dapat diorientasikan dengan suatu arah, dan arah ini berbeda-beda, tergantung pada perubahan intensitas. Gambar 2.1 memperlihatkan model edge dalam ruang satu dimensi.



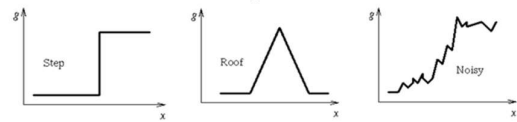
Gambar 2.6 Citra dengan Beberapa Tingkat Resolusi

Menurut Munir [7] ada tiga macam tepi (edge) yang terdapat di dalam citra digital seperti yang diperlihatkan pada gambar 2.9. Ketiga macam edge tersebut adalah:

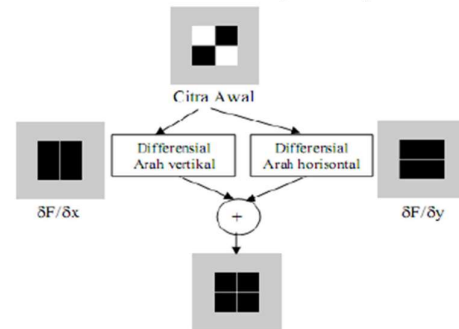
1. Tepi curam
Di mana tepi terbentuk karena perubahan intensitas yang tajam, berkisar 90°.
2. Tepi landai
Di mana tepi lebar, sudut arah kecil. Terdiri dari sejumlah tepi-tepi lokal yang lokasinya berdekatan.
3. Tepi yang mengandung noise
Di mana biasanya dilakukan operator image enhancement terlebih dahulu, misalnya Operator Gaussian yang berfungsi untuk menghaluskan citra.

Edge detection pada suatu citra memiliki tujuan sebagai berikut [3]:

- a. Menandai bagian yang menjadi detil citra.
- b. Memperbaiki detil citra yang kabur karena *error* atau efek proses akuisisi.



Gambar 2.7 Jenis-jenis Tepi



Gambar 2.8 Proses Edge Detection Citra

F. Konvolusi

Edge detection merupakan salah satu proses pengolahan citra yang menggunakan filter, yang dilakukan melalui metode konvolusi. Konvolusi dinyatakan dalam matriks, dimana setiap elemen matriks dinamakan koefisien konvolusi. Operasi konvolusi bekerja dengan menggeser kernel piksel per piksel, yang hasilnya disimpan dalam matriks baru. Konvolusi 2 fungsi $f(x)$ dan $g(x)$ diperlihatkan dengan rumus berikut:

$$f(x) * g(x) = \int_{-\infty}^{\infty} f(\alpha)g(x-\alpha)d\alpha$$

atau fungsi diskritnya:

$$f(x) * g(x) = \sum f(\alpha)g(x-\alpha), \text{ dimana } \alpha = \text{peubah bantu.}$$

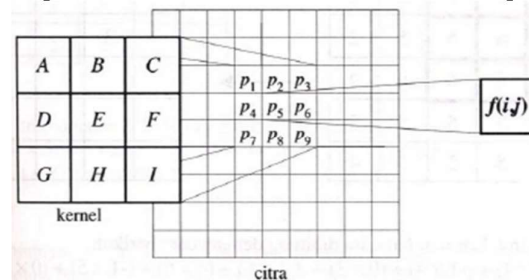
Pada konvolusi 2D fungsi malarnya dapat dihitung dengan persamaan:

$$f(x,y) * g(x,y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(a,b)g(x-a,y-b)dad b$$

Sedangkan fungsi diskritnya dihitung dengan persamaan:

$$f(x,y) * g(x,y) = \sum_{-\infty}^{\infty} \sum_{-\infty}^{\infty} f(a,b)g(x-a,y-b)$$

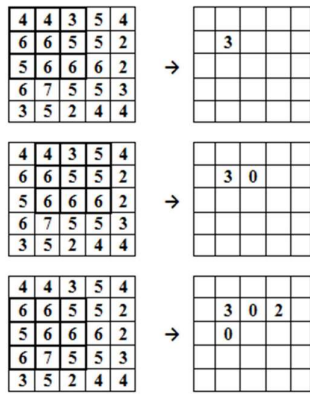
merupakan convolution mask/filter/kernel atau template.



Gambar 2.9 Proses Konvolusi

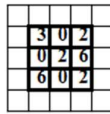
$$f(x) = \begin{bmatrix} 4 & 4 & 3 & 5 & 4 \\ 6 & 6 & 5 & 5 & 2 \\ 5 & 6 & 6 & 6 & 2 \\ 6 & 7 & 5 & 5 & 3 \\ 3 & 5 & 2 & 4 & 4 \end{bmatrix} \quad g(x,y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Gambar 2.10 Matriks Citra dan Kernel Sebelum Konvolusi



dan seterusnya

Gambar 2.11 Tahapan Proses Pembentukan Konvolusi



Gambar 2.12 Hasil Konvolusi Citra dan Kernel

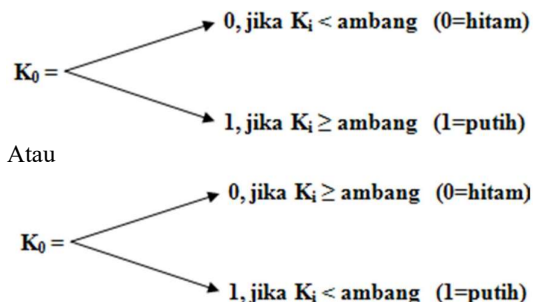
Proses perkalian matriks pada setiap titik pixel yang bersesuaian pada matriks citra dengan matriks kernel. Seperti yang bisa diperhatikan pada Gambar, hasil akhir dari proses konvolusi mengalami pengurangan dua pixel secara horizontal maupun vertikal. Hal ini terjadi karena matriks kernel tidak bisa mengakomodasikan perhitungan pada pixel-pixel paling tepi dari citra awal. Walaupun demikian, pixel-pixel paling tepi dianggap tidak mengandung informasi yang cukup penting sehingga penghilangan pixel-pixel tersebut dari citra hasil konvolusi tidak akan memberikan pengaruh yang signifikan. Dalam konvolusi terdapat dua kemungkinan yang jika ditemukan, diselesaikan dengan cara berikut, yaitu:

Untuk hasil konvolusi dengan nilai negatif, nilainya dijadikan nol (0). Jika hasil konvolusi lebih besar dari derajat keabuan maksimum, maka nilai diubah menjadi nilai maksimum derajat keabuan.

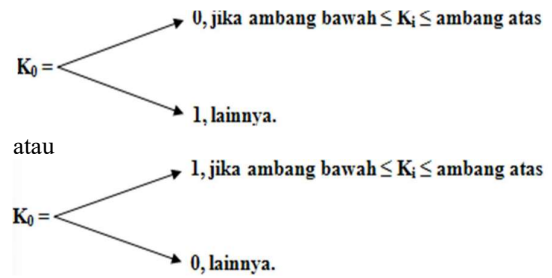
G. Thresholding

Operasi thresholding merupakan salah satu operasi yang termasuk ke dalam operasi titik dalam pengolahan citra digital. Operasi ini digunakan untuk mengubah citra dengan format skala keabuan yang memiliki kemungkinan nilai lebih dari 2 ke citra biner yang memiliki 2 buah nilai (yaitu 0 dan 1). Thresholding terdiri dari dua jenis, yaitu:

- Thresholding Tunggal Thresholding tunggal merupakan proses thresholding yang hanya memiliki sebuah nilai ambang batas. Fungsi GST-nya:



- Thresholding Ganda Memiliki ambang batas atas dan batas bawah. Dilakukan untuk menampilkan titik-titik yang memiliki rentang nilai skala keabuan tertentu. Fungsi GST-nya:



H. Metode Edge Detection Canny

Metode Edge Detection Canny adalah sebuah algoritma yang digunakan untuk mendeteksi tepi pada citra digital. Algoritma ini dikembangkan oleh John F. Canny. Algoritma ini terdiri dari beberapa tahap, yaitu

- Noise Reduction

Tahap pertama adalah mengurangi noise pada citra dengan menggunakan filter Gaussian 5x5.

- Finding Intensity Gradient of the Image

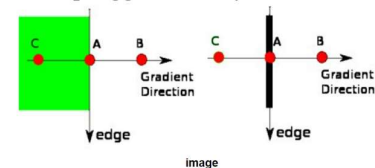
Citra yang telah dihaluskan kemudian difilter dengan kernel Sobel pada arah horizontal dan vertikal untuk mendapatkan turunan pertama pada arah horizontal (G_x) dan vertikal (G_y). Dari kedua citra ini, dapat ditemukan gradien tepi dan arah untuk setiap piksel menggunakan persamaan seperti berikut;

$$Edge_Gradient (G) = \sqrt{G_x^2 + G_y^2}$$

$$Angle (\theta) = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

- Non-Maximum Suppression

Tahap ini bertujuan untuk menghilangkan piksel yang tidak penting pada tepi. Pada setiap piksel, akan diperiksa apakah piksel tersebut merupakan maksimum lokal pada tetangganya pada arah gradien. Jika iya, maka piksel tersebut akan dipertahankan, jika tidak, maka akan dihapus. Berikut contoh penggambarannya;

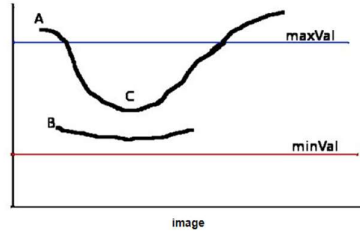


Gambar 2.13 Diagram Non-Maximum Suppression

- Hysteresis Thresholding

Tahap terakhir adalah mengklasifikasikan piksel menjadi tepi atau bukan tepi. Hal ini dilakukan dengan menggunakan dua nilai threshold, yaitu minVal dan maxVal. Piksel yang memiliki gradien intensitas lebih besar dari maxVal pasti merupakan tepi, sedangkan piksel yang memiliki gradien intensitas lebih kecil dari minVal pasti bukan tepi dan akan dihapus. Piksel yang memiliki gradien intensitas antara kedua nilai threshold akan diklasifikasikan sebagai tepi atau bukan tepi berdasarkan keterhubungannya dengan piksel tepi

yang pasti. Dapat dilihat penggambarannya sebagai berikut;



Gambar 2.14 Diagram Hysteresis Thresholding

Dalam OpenCV, metode Canny dapat digunakan dengan menggunakan fungsi `cv2.Canny()`. Fungsi ini menerima beberapa parameter, antara lain

- Image**
Citra input yang akan dikenai filter Canny
- T_lower**
Nilai threshold bawah pada hysteresis thresholding
- T_upper**
Nilai threshold atas pada hysteresis thresholding
- aperture_size**
Ukuran aperture pada filter Sobel
- L2Gradient**
Boolean parameter yang digunakan untuk menghitung gradien tepi dengan lebih presisi.

Berikut contoh gambar yang menggunakan edge detection metode canny;



Gambar 2.15 Contoh Canny operator

1. Metode Edge Detection Laplace

Operator Laplace adalah salah satu operator yang digunakan dalam pengolahan gambar dan penglihatan komputer untuk mendeteksi tepi, perubahan tajam dalam intensitas piksel, atau fitur citra lainnya. Ini mengukur gradien kedua dari intensitas piksel dalam gambar dan sering digunakan untuk mengungkapkan di mana perubahan yang signifikan dalam gambar terjadi. Operator Laplace adalah alternatif yang umum digunakan selain operator deteksi tepi seperti Sobel atau Prewitt. Operator Laplace didefinisikan menggunakan persamaan berikut;

$$\text{Laplace}(f) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Operator Laplace 2D adalah operator gradien kedua yang digunakan untuk mengukur perubahan intensitas piksel dalam dua arah, yaitu horisontal dan vertikal. Ini digunakan untuk menyoroti tepi yang ditandai oleh perubahan tajam dalam intensitas piksel. Representasi operator Laplace 2D dalam citra adalah kernel atau matriks persegi yang

diterapkan pada gambar melalui operasi konvolusi. Operasi konvolusi dengan kernel ini mengukur gradien kedua dalam kedua arah dan menghasilkan citra gradien kedua yang menyoroti tepi. Berikut contoh gambar yang menggunakan edge detection metode laplace;



Gambar 2.16 Contoh Laplace operator

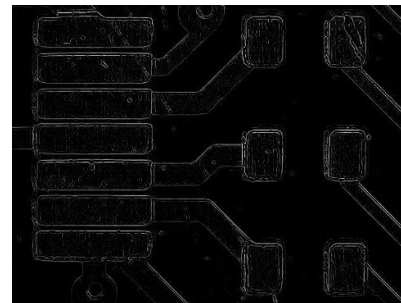
J. Metode Blur Detection Variance of Laplacian

Variance of Laplacian adalah metode untuk mendeteksi blur pada gambar. Metode ini bekerja dengan menghitung variance dari hasil *Laplacian* sebuah gambar. Jika gambar tajam, maka gambar *Laplacian* akan memiliki *variance* yang tinggi. Hal ini karena gambar yang tajam akan memiliki perubahan intensitas piksel yang tajam dalam persebaran yang besar. Sebaliknya, jika gambar buram, maka *variance* dari gambar *Laplacian* akan rendah karena perubahan intensitas pixel yang tajam memiliki persebaran yang rendah.[10]

III. PEMBAHASAN HASIL

Pada penelitian studikusus ini pertama – tama diambil sekumpulan gambar PCB dengan tingkatan blur yang berbeda-beda. Kumpulan gambar tersebut dijalankan kedalam 3 buah program yang menggunakan metode berbeda-beda untuk mendeteksi blur dalam gambar

A. Metode Edge Detection Laplace



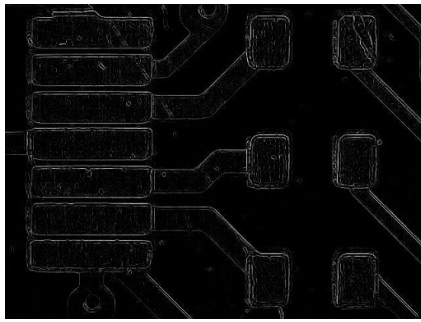
Gambar 3.1 Hasil Edge Detection Laplace

```
(\viskom\test\WIN 20230125_20_22_02_Pro.jpg', 9.6485546875, 'Not blurry')
(\viskom\test\WIN 20230125_20_22_21_Pro.jpg', 9.108597085208335, 'Not blurry')
(\viskom\test\WIN 20230125_21_04_06_Pro.jpg', 9.023929036458334, 'Not blurry')
(\viskom\test\WIN 20230125_20_40_29_Pro.jpg', 8.631230468750001, 'Not blurry')
```

Gambar 3.2 Pengurutan Gambar Hasil Edge Detection Laplace

Melalui metode di atas didapatkan nilai kekuatan dari *edge* atau tepi dari semua gambar yang digunakan dalam program. Nilai ini kemudian digunakan untuk mengurutkan gambar, dengan nilai yang terbesar menunjukkan bahwa gambar tersebut memiliki *edge* atau tepi yang paling kuat. Terlihat pada Gambar 3.1, gambaran *edge* atau tepi dari gambar yang memiliki nilai kekuatan *edge* paling besar.

B. Metode Blur Detection Variance of Laplacian



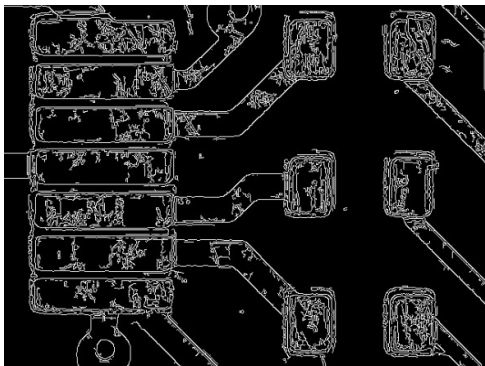
Gambar 3.3 Hasil Edge Detection Laplace pada Metode Blur Detection Variance of Laplacian

'viskom/test\WIN_20230125_20_22_02_Pro.jpg',	752.9487648381552,	'Not blurry')
'viskom/test\WIN_20230125_20_22_21_Pro.jpg',	669.3399612477514,	'Not blurry')
'viskom/test\WIN_20230125_21_04_06_Pro.jpg',	648.4448277918394,	'Not blurry')
'viskom/test\WIN_20230125_20_40_29_Pro.jpg',	610.163377522405,	'Not blurry')
'viskom/test\WIN_20230125_20_40_42_Pro.jpg',	560.3495623363814,	'Not blurry')

Gambar 3.4 Pengurutan Gambar Hasil Blur Detection Variance of Laplacian

Melalui metode ini, didapatkan *variance* atau persebaran dari hasil laplace gambar. Hal ini berkorelasi dengan metode sebelumnya yang menggunakan laplace untuk menghitung tingkat kekuatan dari *edge*, Sedangkan metode *variance of laplace* menggunakan Langkah tambahan yaitu menghitung *variance* atau persebarannya. Semakin besar *variance* nya artinya gambar tersebut memiliki perubahan *edge* secara tajam dalam persebaran yang besar, yang berarti gambar tersebut memiliki ketajaman yang tinggi. Sebaliknya, gambar dengan tingkat blur yang tinggi akan memiliki *variance* yang rendah karena perubahan tepi secara tajam memiliki persebaran yang rendah.

C. Metode Edge Detection Canny



Gambar 3.5 Hasil Edge Detection Canny

'viskom/test\WIN_20230125_20_22_02_Pro.jpg',	27.264746093750002,	'Not blurry')
'viskom/test\WIN_20230125_21_04_06_Pro.jpg',	25.884326171875003,	'Not blurry')
'viskom/test\WIN_20230125_20_22_21_Pro.jpg',	25.270068359375003,	'Not blurry')
'viskom/test\WIN_20230125_20_40_29_Pro.jpg',	24.730517578125003,	'Not blurry')
'viskom/test\WIN_20230125_20_40_42_Pro.jpg',	24.337060546875,	'Not blurry')

Gambar 3.6 Pengurutan Gambar Hasil Edge Detection Canny

Metode canny dapat menghasilkan *edge* yang lebih jelas dan lebih tipis dibandingkan metode laplace. Metode canny menggunakan local maxima dalam arah gradient untuk menentukan *edges*, sehingga *edge* yang dihasilkan lebih tipis dibandingkan metode laplace. Selain itu, metode canny menggunakan *thresholding* untuk menentukan "*sure-edge*" atau *edge* pasti yang memiliki nilai intensitas gradient diatas

threshold maksimum. Semua *edge* yang berhubungan dengan dengan "*sure-edge*" akan dianggap sebagai *edge* hasil akhir.

Metode canny digunakan untuk mendeteksi blur menggunakan prinsip yang sama dengan metode laplace, dimana kekuatan *edge* nya yang akan diperhitungkan. Gambar dengan kekuatan *edge* besar berarti memiliki ketajaman gambar yang lebih besar. Hal ini didasarkan pada sifat dari blur yang menghaluskan tepi sehingga gambar dengan blur yang tinggi akan memiliki tepi yang cukup lemah.

IV. KESIMPULAN

Penelitian ini bertujuan untuk mengevaluasi kualitas gambar dengan berbagai tingkat blur menggunakan metode deteksi tepi (*edge detection*) dengan OpenCV. Tiga metode deteksi tepi yang digunakan dalam penelitian ini adalah Laplace, Variance of Laplacian, dan Canny.

Metode Laplace digunakan untuk mengukur kekuatan tepi (*edge*) dalam gambar. Hasil dari metode ini digunakan untuk mengurutkan gambar, dengan nilai tertinggi menunjukkan gambar yang memiliki tepi terkuat. Sementara itu, Metode Variance of Laplacian digunakan untuk mengukur persebaran (*variance*) hasil Laplace gambar. Semakin besar *variance*, semakin tajam tepi gambar. Sebaliknya, gambar dengan tingkat blur tinggi akan memiliki *variance* yang rendah. Dan Kemudian untuk metode Canny menghasilkan tepi gambar yang lebih jelas dan tipis dibandingkan dengan metode Laplace. Ini karena Canny menggunakan local maxima dalam arah gradient untuk menentukan tepi, serta menggunakan *thresholding* untuk menentukan tepi pasti. Dari ketiga metode tersebut, kita dapat menyimpulkan beberapa hal:

- Metode Canny menghasilkan tepi gambar yang lebih tajam dan jelas dibandingkan dengan metode Laplace.
- Metode Variance of Laplacian memberikan informasi tambahan tentang sejauh mana tepi dalam gambar tersebar, yang dapat digunakan untuk mengidentifikasi tingkat ketajaman gambar.
- Gambar dengan nilai kekuatan tepi yang tinggi mengindikasikan gambar dengan tingkat ketajaman yang lebih besar, sedangkan gambar dengan blur tinggi memiliki nilai kekuatan tepi yang lebih rendah.

Kesimpulannya, sistem model pengolahan citra menggunakan OpenCV pada perangkat digital yang sudah dibuat berhasil menggunakan metode deteksi tepi untuk mengevaluasi kualitas gambar dengan berbagai tingkat blur. Metode Canny lebih efektif dalam menghasilkan tepi yang tajam, sementara Variance of Laplacian dapat memberikan informasi tambahan tentang tingkat ketajaman gambar. Hasil ini dapat digunakan dalam pemilihan gambar atau pengolahan gambar lebih lanjut tergantung pada tujuan analisis atau penggunaan yang diinginkan.

REFERENCES

- [1] Adrian Rosebrock et al. "Find distance from camera to object/marker using Python and OpenCV", 2015.
- [2] B. M U, H. Raghuram and Mohana, "Real Time Object Distance and Dimension Measurement using Deep Learning and OpenCV" 2023 Third International Conference on Artificial Intelligence and Smart Energy (ICAIS), Coimbatore, India, 2023, pp. 929-932, Doi:

- [3] Chen.W, et al. "An improved edge detection algorithm for depth maps in painting", *Optics and Lasers in Engineering*, 2014, pp.69–77.
- [4] Febriani. "Analisis penelusuran tepi citra menggunakan deteksi tepi sobel dan canny". Dalam Febriani dan Lussiana, E. T. P. (eds). *Seminar Ilmiah Nasional Komputer dan Sistem Intelijen (KOMMIT 2008)*: hal 462-466. Depok: Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma.
- [5] Hestningsih, I. *Pengolahan Citra*. 2008.
- [6] Indira, M. "Perbandingan metode pendeteksi tepi". Dalam Indira, M, dkk. (eds). *Seminar Ilmiah Nasional Komputer dan Sistem Intelijen (KOMMIT 2008)*: hal 366-373. Depok: Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma.
- [7] Muthu Krishnan R et al. "Edge Detection Techniques for Picture Segmentation"2011.
- [8] Shafry, M., Rahim, M., Isrozaidi, N., Ismail, N., dan Idris, M. A. S. "The use of hybrid technique: thresholding and edge detection for identifying river from aerial photo". *Jurnal Teknologi*, 41(B) Dis. 2004: 1–20. Universiti Teknologi Malaysia.
- [9] Sigit, R, et al. *Step By Step Pengolahan Citra Digital*. 2005. Yogyakarta : Andi Offset.
- [10] Taurisna, A. "Analisis pengaruh kualitas resolusi citra terhadap kinerja metode pendeteksi tepi". 2009. Medan : Departemen Ilmu Komputer Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Sumatera Utara.